



Codex CLI · Cheat Sheet

Quick reference for the terminal UI, codex exec, config, MCP, skills, subagents, hooks & rules.

cmd command

Key

keyboard

⚠ caution

📌 START HERE

`codex` Open interactive TUI in current dir
`codex "prompt"` Start session with initial instruction
`codex exec "prompt"` Non-interactive run for scripts / CI
`codex resume` · `--last` Pick or resume latest session
`codex fork --last` Fork latest session into a new thread
`codex login` Sign in with ChatGPT OAuth
`codex login --device-auth` Headless / remote sign-in
`codex login status` Exits 0 when signed in
`codex update` Apply CLI update when supported
`codex completion <shell>` bash / zsh / fish / powershell

> INTERACTIVE TUI

`@` Fuzzy file search; insert path into prompt
`/` Run a slash command
`!` cmd Run shell under current sandbox / approvals
`Ctrl` + `C` or `/exit` Exit session
`Ctrl` + `L` Clear / redraw screen
`Ctrl` + `O` · `/copy` Copy latest output
`Ctrl` + `R` Search prompt history
`Ctrl` + `G` Open prompt in \$EDITOR
`Tab` while running Queue follow-up prompt
`Enter` while running Inject into current turn
`Esc` `Esc` Edit previous user message

⚡ SLASH COMMANDS

`/status` Session, workspace, session id
`/permissions` Switch approval / sandbox mode
`/clear` · `/compact` Clear or summarize context
`/resume` · `/fork` · `/side` Branch threads
`/model` Switch model & reasoning
`/fast on|off|status` Toggle Fast mode
`/plan` Enter Plan mode
`/review [focus]` Local code-review preset
`/mcp` · `/agent` · `/apps` Inspect / switch
`/plugins` · `/skills` · `$skill` Browse / invoke
`/theme` · `/mention` · `/init`
`/personality` · `/feedback` · `/experimental`

⚙ GLOBAL FLAGS

`-m` , `--model <m>` Override model
`-p` , `--profile <n>` Load [profiles.<n>]
`-c key=value` One-off TOML override
`-C` , `--cd <path>` Set workspace root
`--add-dir <p>` Extra writable root (repeatable)
`-i` , `--image <p,p>` Attach images to first prompt
`-s` , `--sandbox` read-only / workspace-write / danger-full-access
`-a` , `--ask-for-approval` untrusted / on-request / never
`--search` Live web search this run
`--oss` Local OSS provider (Ollama / configured)
`--enable` / `--disable <feat>` Force flag
`--no-alt-screen` Disable alternate-screen render
`--remote ws://...` Connect TUI to app server
⚠ `--yolo` Bypass approvals & sandbox

🖨 CODEX EXEC

`codex exec "prompt"` stderr=progress, stdout=answer
`codex exec -` Read prompt from stdin
Pipe stdin Treats command output as extra context
`exec resume --last "p"` Continue last exec session
`--json` JSONL events: turns, items, tools, plans
`-o <file>` Save final assistant message
`--output-schema <f>` Enforce JSON Schema
`--ephemeral` Don't persist session rollout
`--ignore-user-config` · `--ignore-rules`
`--skip-git-repo-check` Run outside Git repo
`CODEX_API_KEY=...` One-shot API-key auth

🛡 APPROVALS & SANDBOX

`read-only` No edits or commands without approval
`workspace-write` Edit workspace + `--add-dir` roots
`danger-full-access` No sandbox — isolated only
`untrusted` Prompt before untrusted commands
`on-request` Prompt on escalation request
`never` Don't prompt — non-interactive runs

Default: `--sandbox workspace-write --ask-for-approval on-request` `./git/` , `./codex/` , `./agents/` stay read-only.

🔌 MODELS, SPEED, SEARCH

`gpt-5.5` recommended · fallback `gpt-5.4`
`/model` Switch in TUI
`codex --model gpt-5.5` Per-run override
`/fast on|off|status` Fast mode
`service_tier = "fast"` Persist Fast mode
`gpt-5.3-codex-spark` Pro research-preview
`web_search = cached` / `live` / `disabled`

📖 AGENTS.MD GUIDANCE

`~/codex/AGENTS.md` Global personal
`~/codex/AGENTS.override.md` Global override
`<repo>/AGENTS.md` Repo-level shared
`<sub>/AGENTS.override.md` Subtree override
Discovery: global → root → CWD. Closer files & `.override.md` win. Use `/init` to scaffold.

🔧 CONFIG & PRECEDENCE

`~/codex/config.toml` User config
`./codex/config.toml` Project (trusted only)
`/etc/codex/config.toml` System (Unix)
`$CODEX_HOME` Override `~/codex` root
`~/codex/auth.json` Credentials — treat as secret
`~/codex/history.jsonl` · `log/` Local state
Order: CLI / `-c` > profile > project > user > system > defaults.

🔌 MCP

`/mcp` View active servers in TUI
`codex mcp list` · `get <n>` Inspect
`codex mcp add <n> -- <cmd...>` Add stdio
`codex mcp add <n> --url https://...` Add HTTP
`codex mcp remove <n>` Remove server
`codex mcp login <n>` OAuth on HTTP servers
`codex mcp-server` Run Codex itself as MCP server

Keys: `command/args/env/cwd` , `url/bearer_token_env_var` , `enabled` , `required` , `enabled_tools`.

✂ SKILLS, PLUGINS & SUBAGENTS

`$skill-name` Explicitly invoke a skill
`/skills` · `$skill-creator` Browse / scaffold
`$skill-installer <n>` Install curated skill
`/plugins` Browse / install / toggle
`codex plugin marketplace add|upgrade|remove`
`./agents/skills/<n>/SKILL.md` Repo skill
`~/agents/skills/<n>/SKILL.md` User skill
Built-in subagents: `default` , `worker` , `explorer`
`/agent` Switch threads · custom in
`~/codex/agents/*.toml`

📖 RULES & HOOKS

`~/codex/rules/default.rules` User rules
`<repo>/codex/rules/*.rules` Project (trusted)
`execpolicy check --rules f -- cmd` Test decisions
Decisions: `allow` · `prompt` · `forbidden`
`[features].codex_hooks = true` Enable hooks
Lifecycle: `SessionStart` , `PreToolUse` , `PostToolUse` , `PermissionRequest` , `UserPromptSubmit` , `Stop`
Hooks read JSON from stdin. matcher is regex (e.g. Bash , `apply_patch` , `mcp_server__tool`).

☁ SESSIONS, REMOTE & CLOUD

`codex resume [--last|<ID>|--all]` Pick / resume
`codex fork [--last]` Branch a session
`codex app-server --listen ws://...:4500`
`codex --remote ws://...:4500` Connect TUI
`--ws-auth capability-token --ws-token-file <f>`
`codex cloud` Browse cloud tasks
`codex cloud exec --env ID "p"` Submit task
`codex apply <TASK_ID>` Apply cloud diff
Use SSH forwarding or TLS+auth for non-local app servers.

📋 WORKFLOW PATTERNS

Explain: Read `@a` `@b` — flow + gotchas

Fix bug: repro + constraints + suspects + done-when

Add tests: happy path & edges for `@file`

From image: `codex -i shot.png "Build this UI..."`

Review: `/review` Focus on security & edges

CI:

`codex exec --json --output-schema s.json -o r.json`

Prompt template: Goal + Context + Constraints + Done-when. Move durable rules to `AGENTS.md` , methods to skills, live context to MCP, automation to `codex exec` / `cloud`.